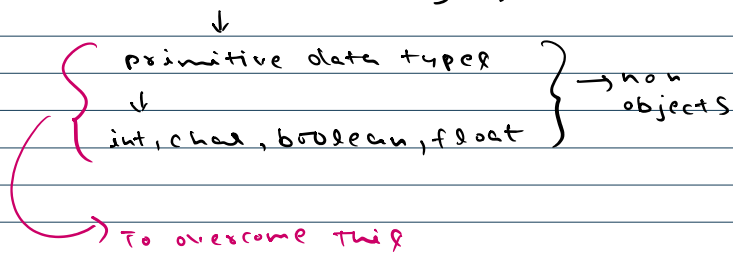
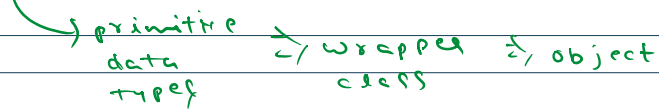


Java → object oriented Language



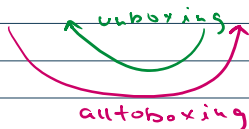
↓

wrapper classes ⇒ they are used to wrap primitive data types. → encapsulate

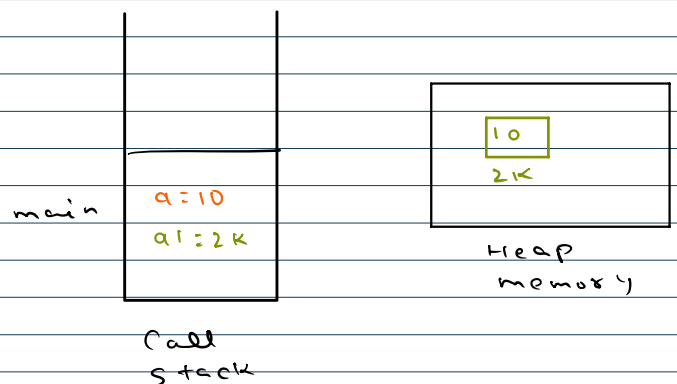


Primitive ↔ wrapper class

<u>Primitive</u>	<u>Wrapper</u>
int	Integer
byte	Byte
short	Short
long	Long
float	Float
double	Double
char	Character
boolean	Boolean



```
public class WrapperClassesDemo {
    public static void main(String args[]) {
        int a = 10;
        Integer a1 = 10; // auto-boxing
        System.out.println("Hello Akarsh!");
    }
}
```



Cache Range

Integer, Byte, Short, Long ⇒ -128 to 127

Character ⇒ 0 to 127

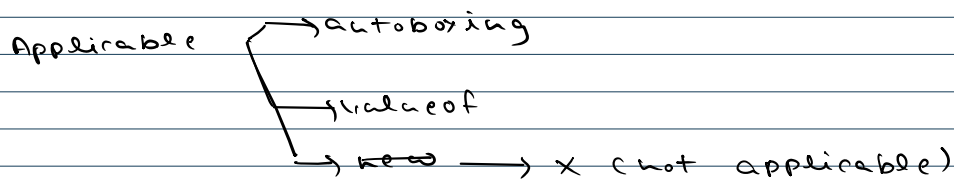
Boolean ⇒ true/false

Float, Double ⇒ never in cache, always new instance

number $\Rightarrow 0$ to 10^9

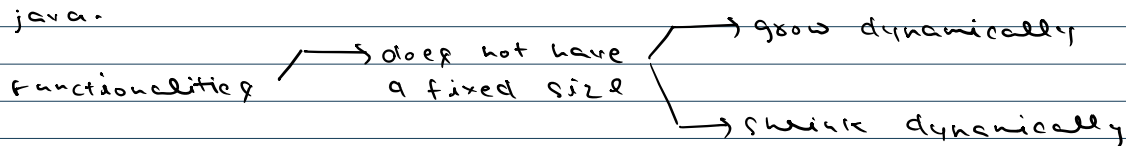
Boolean $\Rightarrow$ true/false

Float, Double $\Rightarrow$ never in cache, always new instance



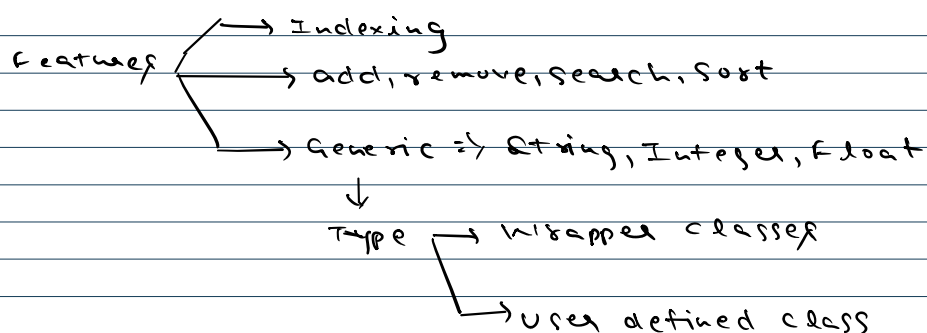
Array List

It is a resizable array and its implementation is provided in java.



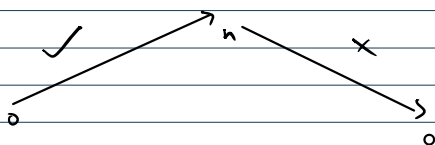
ArrayList $\rightarrow$ class $\rightarrow$ object

$\downarrow$
Non-primitive
data type
 $\downarrow$
Heap memory



For each loop

Reverse $\rightarrow$ X



$$\text{new cap} = \text{old cap} + (\text{old} / 2) + 1 \Rightarrow 10.5 \times$$

$$10 \Rightarrow 16 \quad \left(10 + \left(\frac{10}{2} \right) + 1 \right) = (10 + 5 + 1) = 16$$

Print all substrings length wise

n k n x ch

Print all substrings length wise

akars h

len 1 $\Rightarrow$ a, k, a, r, s, h

len 2 $\Rightarrow$ ak, ka, ar, rs, sh

len 3 $\Rightarrow$ aka, kar, ars, rsh

len 4 $\Rightarrow$ akar, kars, arsh

len 5 $\Rightarrow$ akars, kars h

len 6 $\Rightarrow$ akars h

0 1 2 3 4 5
a k a r s h

i j
↑ ↑
len

0-1 a
1-2 k
2-3 a
3-4 r
4-5 s
5-6 h
j-i: 1
(len)

0-2 ak
1-3 ka
2-4 ar
3-5 rs
4-6 sh
j-i: 2
(len)

0-3 aka
1-4 kar
2-5 ars
3-6 rsh
j-i: 3
(len)

0-4 akar
1-5 kars
2-6 arsh
j-i: 4
(len)

0-5 akars
1-6 kars h
j-i: 5
(len)

0-6 akars h
j-i: 6
(len)

j-i: len

j-len = i

i = j-len

```
for (len = 1; len <= s.length(); len++) {
    for (int j = len; j <= s.length(); j++) {
        int i = j - len;
        SOP(substring(i, j));
    }
}
```